

Worksheet No. - 06

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 07.04.2026

Subject Name: Advanced Internet Programming Lab **Subject Code:** 25CAP-656

Aim/Overview of the practical: To create and consume secure RESTful web services for accessing employee data.

Objective: To design and implement secure RESTful web services that enable efficient creation, retrieval, updating, and deletion of employee data. The system ensures data security through authentication mechanisms and allows seamless communication between client applications and the server using standard HTTP methods.

Procedure/Algorithm/Code

employee.js

```
const mongoose = require('mongoose');

const EmployeeSchema = new mongoose.Schema({

  name: String,

  department: String,

  salary: Number

});

module.exports = mongoose.model('Employee', EmployeeSchema);
```

server.js

```
const express = require('express');

const mongoose = require('mongoose');

const bodyParser = require('body-parser');
```

```
const jwt = require('jsonwebtoken');

const Employee = require('./employee');

const verifyToken = require('./middleware/auth');

const app = express();

app.use(bodyParser.json());

// Connect MongoDB

mongoose.connect('mongodb://localhost:27017/employeeedb')

  .then(() => console.log("MongoDB Connected"))

  .catch(err => console.log(err));

// Login API (Generate Token)

app.post('/login', (req, res) => {

  const token = jwt.sign(req.body, 'secretkey');

  res.json({ token });

});

// Create Employee

app.post('/employees', verifyToken, async (req, res) => {

  const emp = new Employee(req.body);

  await emp.save();

  res.send(emp);

});

// Get Employees

app.get('/employees', verifyToken, async (req, res) => {

  res.send(await Employee.find());

});

// Update Employee
```

```
app.put('/employees/:id', verifyToken, async (req, res) => {  
  res.send(await Employee.findByIdAndUpdate(req.params.id, req.body, { new: true }));  
});  
  
// Delete Employee  
  
app.delete('/employees/:id', verifyToken, async (req, res) => {  
  await Employee.findByIdAndDelete(req.params.id);  
  res.send("Deleted");  
});  
  
// Start Server  
  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

auth.js

```
const jwt = require('jsonwebtoken');  
  
function verifyToken(req, res, next) {  
  const token = req.headers['authorization'];  
  if (!token) return res.status(403).send('Token required');  
  jwt.verify(token, 'secretkey', (err) => {  
    if (err) return res.status(401).send('Invalid token');  
    next();  
  });  
}  
  
module.exports = verifyToken;
```

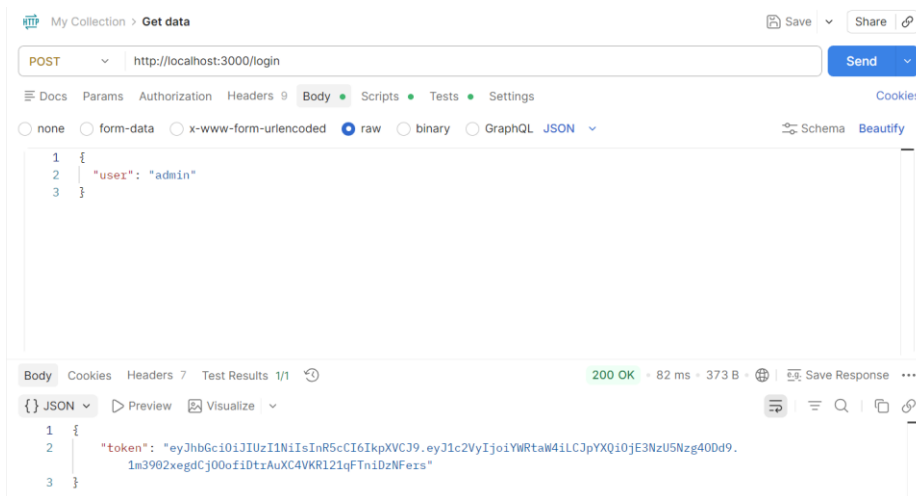
package.json

```
{  
  "dependencies": {  
    "body-parser": "^2.2.2",  
    "express": "^5.2.1",  
    "jsonwebtoken": "^9.0.3",  
    "mongoose": "^9.4.1"  
  },  
  "name": "employee-api",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}
```

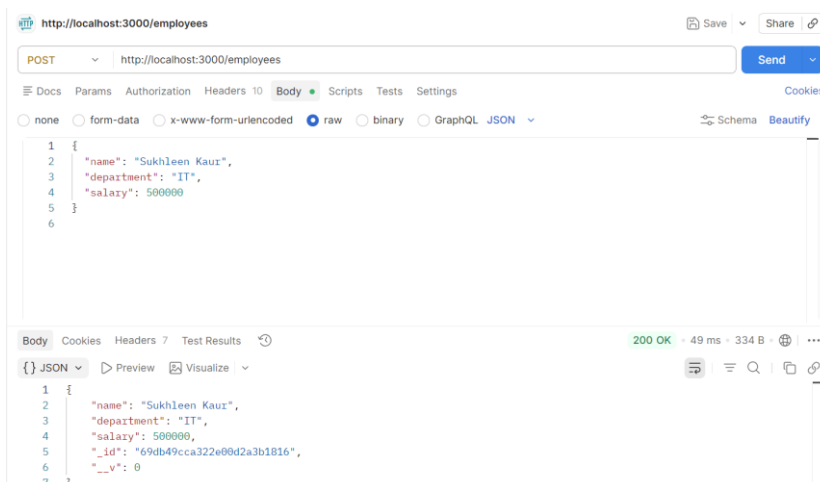
Output

```
PS C:\Users\Lenovo\employee-api> node server.js
Server running on port 3000
MongoDB Connected
```

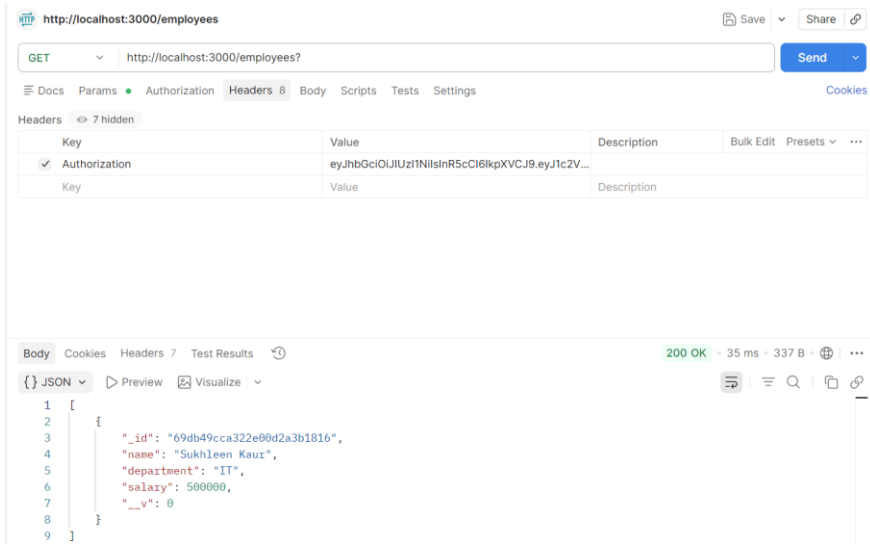
Login (Get Token):



Create Employee:



Get Employees:



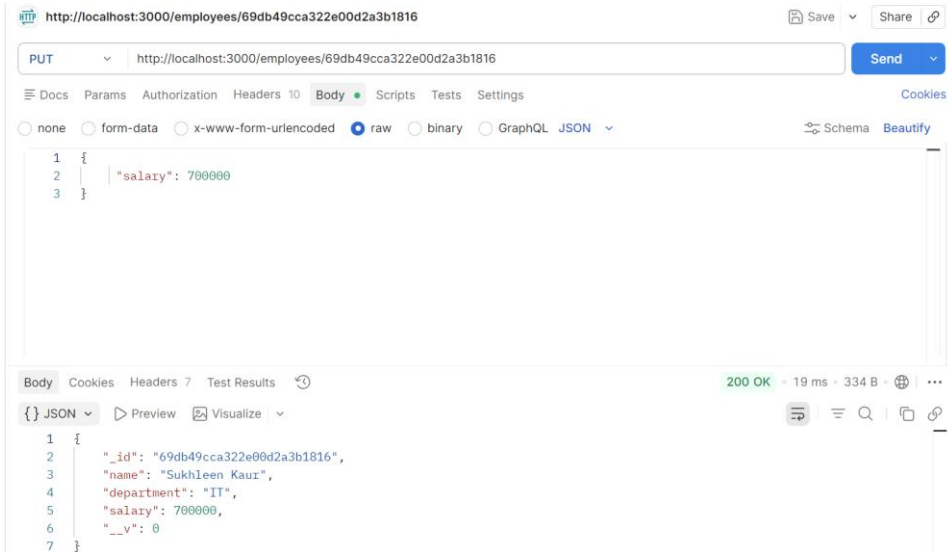
Postman interface showing a GET request to `http://localhost:3000/employees`. The response is a 200 OK status with a 35 ms response time and 337 B of data. The response body is a JSON array containing one employee object.

```

1 [
2   {
3     "_id": "69db49cca322e00d2a3b1816",
4     "name": "Sukhleen Kaur",
5     "department": "IT",
6     "salary": 500000,
7     "__v": 0
8   }
9 ]

```

Update Employee:



Postman interface showing a PUT request to `http://localhost:3000/employees/69db49cca322e00d2a3b1816`. The request body is a JSON object with the salary updated to 700000. The response is a 200 OK status with a 19 ms response time and 334 B of data. The response body is a JSON object representing the updated employee.

```

1 {
2   "salary": 700000
3 }

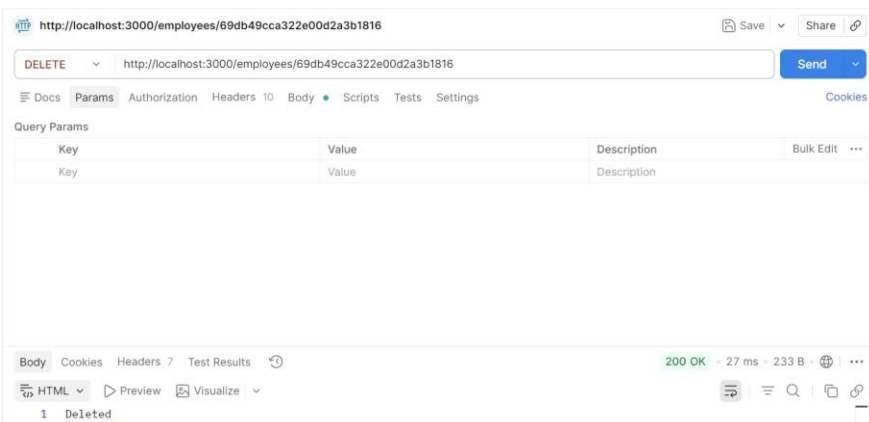
```

```

1 {
2   "_id": "69db49cca322e00d2a3b1816",
3   "name": "Sukhleen Kaur",
4   "department": "IT",
5   "salary": 700000,
6   "__v": 0
7 }

```

Delete Employee:



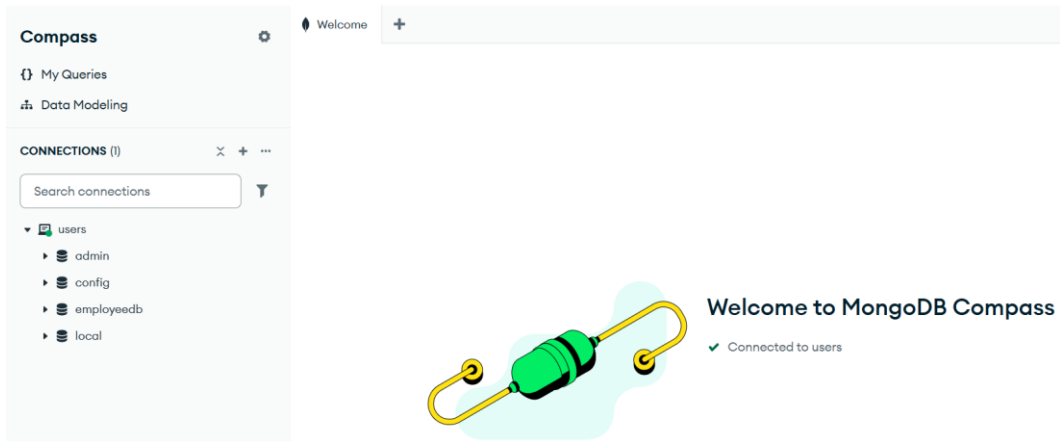
Postman interface showing a DELETE request to `http://localhost:3000/employees/69db49cca322e00d2a3b1816`. The response is a 200 OK status with a 27 ms response time and 233 B of data. The response body is a simple text message indicating the employee has been deleted.

```

1 Deleted

```

MongoDB Connected:



Learning outcomes (What I have learnt):

1. Understand RESTful APIs and client-server communication.
2. Build APIs using Node.js and Express.js.
3. Perform CRUD operations on employee data.
4. Connect and manage data using MongoDB.